A Uniform Optimum for Short-Interval Square-Difference Masks

A Lean-verified result for a 256-bit raw-table model

GPT-6 astra

10 October 2026

Abstract

Fermat's factorization method searches for a representation of an integer N as a difference of two squares, $N = a^2 - b^2$. A modular filter can reject a candidate a whenever $a^2 - N$ is not a square modulo a chosen integer m. The filter repeats every m positions, so its decisions can be stored in a table with one bit per residue. Which modulus gives the strongest guaranteed filtering when this table is limited to 256 bits?

We measure filtering strength by the largest number of candidates that can pass in any h consecutive positions, taking the worst case over interval starts and inputs coprime to the chosen modulus. We prove that $m = 144$ minimizes this worst-case count among every integer modulus $1 \le m \le 256$, simultaneously for every positive interval length h. For example, at most two candidates pass in any interval of 18 positions, and at most four pass in any interval of 48 positions. We give an explicit formula for the count at modulus 144 and formalize the central optimality theorem in Lean 4. The formal proof checks finite certificates directly from the residue definition, then uses periodicity to cover all lengths. Two Python implementations provide additional computational checks.

The theorem concerns candidate counts under this specific table-storage model; it does not establish a faster factorization algorithm. Earlier work already studies modular filtering and uses modulus 144. Whether the precise all-length optimality theorem is new remains unresolved.

1 Definition and scope

Write $Q_m = \{b^2 \bmod m : b \in \mathbb{Z}\}$ and let $\mathcal{U}_m$ denote the unit residues modulo m. Set $\mathcal{U}_1 = \{0\}$ and $Q_1 = \{0\}$. Define

$$S_m(n) = \{a \bmod m : a^2 - n \bmod m \in Q_m\}, \qquad C_{m,n,s}(h) = \sum_{j=0}^{h-1} \mathbf{1}_{S_m(n)}(s+j),$$

$$F_m(h) = \max_{n \in \mathcal{U}_m,\ 0 \le s < m} C_{m,n,s}(h), \qquad G_B(h) = \min_{1 \le m \le B} F_m(h).$$

Positions are counted with multiplicity when $h > m$. We also use $C_{m,n,s}(0) = F_m(0) = 0$.

The storage charge is exactly m logical bits for a one-bit-per-residue table for the current n. The limit $B = 256$ therefore represents 32 bytes of raw table payload. Metadata, construction workspace, verifier memory, and a cache over all values of n are excluded. This is an explicit uncompressed representation. In fact, every even-m mask has period dividing $m/2$, and parity permits further packing in some cases. No claim of optimal compressed storage follows.

The maxima over n and s give uniform bounds for arbitrary intervals. They need not be attained at the correlated start and length of an actual Fermat factorization search. The verification uses finite residue sets, without accepting an integer to factor or testing any deployed key.

Theorem 1. *For every integer $h \geq 1$ and every integer $1 \leq m \leq 256$,*

$$F_{144}(h) \leq F_m(h).$$

Thus $G_{256}(h) = F_{144}(h)$ for all positive lengths. The assertion allows ties at individual lengths and does not extend the modulus budget.

The paper first gives an analytic description of the modulus-144 family and the finite comparisons used in the proof. Section 6 gives a separate Lean 4 formalization of Theorem 1, including the finite bounds and their extension to arbitrary lengths. The exact envelope formula below retains its analytic proof; it is not a premise assumed by the Lean theorem.

2 An exact formula for the modulus 144

Define two functions on nonnegative integers by

$$B(h) = 2\left\lfloor \frac{h}{24} \right\rfloor + b(h \bmod 24), \qquad Q(h) = 4\left\lfloor \frac{h}{72} \right\rfloor + k(h \bmod 72),$$

where

$$b(r) = \begin{cases} 0 & r = 0, \\ 1 & 1 \leq r \leq 6, \\ 2 & 7 \leq r \leq 23, \end{cases} \qquad k(r) = \begin{cases} 0 & r = 0, \\ 1 & 1 \leq r \leq 2, \\ 2 & 3 \leq r \leq 18, \\ 3 & 19 \leq r \leq 26, \\ 4 & 27 \leq r \leq 71. \end{cases}$$

Lemma 2. *For every integer $h \geq 0$, $F_{144}(h) = \max\{B(h), Q(h)\}$.*

Proof. CRT separates the conditions modulo 16 and 9. The square residues modulo 16 are $0, 1, 4, 9$. Direct substitution gives the complete unit table

$n \pmod{16}$	allowed a
$1, 13$	$1, 7 \pmod 8$
$5, 9$	$3, 5 \pmod 8$
$3, 11$	$2 \pmod 4$
$7, 15$	$0 \pmod 4$

Modulo 9, $n = 2, 5, 8$ gives precisely $a = 0 \pmod 3$. The values $n = 1, 4, 7$ give respectively $a = \pm u \pmod 9$ with $u = 1, 2, 4$.

First combine $a = 0 \pmod 3$ with the modulo-16 patterns. A single class modulo 4 becomes one class modulo 12. A two-class pattern modulo 8 becomes two points, separated by 6, in a period of 24. The latter family has envelope $B(h)$ and dominates the former.

Next combine the two-root pattern modulo 9 with a single class modulo 4. There are two points per period 36, with shorter cyclic separation 4, 8, or 16. The largest envelope is obtained with separation 4. Its thresholds for one through four points within a period of 72 are $1, 5, 37, 41$.

It remains to combine two roots modulo 9 with two classes modulo 8. Translation by 36 exchanges the two possible modulo-8 patterns and preserves the modulo-9 roots. It suffices to intersect $a = \pm 1 \pmod 8$ with $a = \pm u \pmod 9$. The resulting masks are

u	positions in $[0, 71]$	cyclic gaps	least lengths for 1,2,3,4 points
1	$1, 17, 55, 71$	$16, 38, 16, 2$	$1, 3, 19, 35$
2	$7, 25, 47, 65$	$18, 22, 18, 14$	$1, 15, 33, 51$
4	$23, 31, 41, 49$	$8, 10, 8, 46$	$1, 9, 19, 27$

A window containing successive selected points requires one plus the sum of the intervening gaps. Maximizing these three profiles gives $Q(h)$. The preceding single-class-modulo-4 case is dominated, since its thresholds $1, 5, 37, 41$ are no earlier than $1, 3, 19, 27$ and both have four points per period 72. All CRT combinations used here are realized by unit residues n, so the maximum of the remaining two envelopes is exact. □

Corollary 3. *For every integer $h \geq 0$,*

$$F_{144}(h) \leq \frac{h}{12} + \frac{7}{4}.$$

The additive constant $7/4$ is attained.

Proof. In $B(h) - h/12$, complete 24-blocks cancel. The largest remainder excess is $2 - 7/12 = 17/12$. In $Q(h) - h/12$, each complete 72-block reduces the excess by 2. The remainder jumps at $1, 3, 19, 27$, where the excesses are $11/12, 7/4, 17/12, 7/4$. Thus $7/4$ is an upper bound. For $n = 1$ and the three-position interval $-1, 0, 1$, exactly two positions pass, attaining $2 - 3/12 = 7/4$. A second example is $n = 97$, interval $23, \ldots, 49$, with four passing positions and excess $4 - 27/12 = 7/4$. □

3 Finite comparisons and the infinite tail

Put $R_m(n) = |S_m(n)|$ and $\rho_m = \max_{n \in \mathcal{U}_m} R_m(n)/m$.

Lemma 4. *For all $m \geq 1$ and $h \geq 0$, $F_m(h) \geq \lceil h\rho_m \rceil$.*

Proof. Choose a mask of maximum cardinality. In the sum of its window counts over the m starts, every marked residue occurs exactly h times. Its average window count is therefore $h\rho_m$; the maximum is at least its ceiling. □

Lemma 5 (Finite certificate)**.** *Complete enumeration in the stated finite domain establishes:*

(i) $\rho_{144} = 1/12$ *and* $\rho_m \geq 2/21$ *for every* $1 \leq m \leq 256$, $m \neq 144$. *The latter minimum occurs at* 252.

(ii) $F_{144}(h) \leq F_m(h)$ *for every* $1 \leq h \leq 146$ *and* $1 \leq m \leq 256$.

The computational evidence for this lemma is described explicitly in Section 5. As a compact check of (i), the complete list of moduli in this domain with $\rho_m < 1/8$ is

m	144	180	240	252
ρ_m	1/12	1/10	1/10	2/21

The proof does not assume that low density by itself orders short windows.

Proof of Theorem 1. Lemma 5(ii) proves the claim through $h = 146$. For a competing modulus and $h \geq 147$, Lemmas 4 and 5, together with Corollary 3, give

$$F_m(h) \;\geq\; \frac{2h}{21} \;\geq\; \frac{h}{12} + \frac{7}{4} \;\geq\; F_{144}(h).$$

The middle inequality is exactly $h/84 \geq 7/4$, or $h \geq 147$. This covers every positive integer length. For $h > 147$ the middle inequality is strict, proving unique optimality at each $h \geq 148$ as an additional sufficient conclusion. □

The original, prospectively fixed horizon was $1 \leq h \leq 64$. The all-length extension was recorded after the common winner and density gap were observed. The original results were preserved. The extension changed neither the mask definition nor the budget or admissible moduli.

4 What the optimum says about short windows

The exact envelope yields the following original-horizon values:

h	1–2	3–18	19–26	27–48	49–54	55–64
$G_{256}(h)$	1	2	3	4	5	6

.

The least raw-bit budgets attaining these final-budget optima are

h	1	2	3	4	5–6	7–8	9–12	13–64
least budget	1	4	3	4	9	16	36	144

.

Individual-length ties are retained in the certificate. At $h = 20$ the optimal moduli are $144, 180$; at $h = 32$ they are $144, 180, 252$. Thus the common optimum need not be the cheapest choice for a particular short length.

There is also an elementary budget-independent consequence. For any modulus and $h \geq 3$, choosing $n = 1$ admits both $a = -1$ and $a = 1$ in a three-position interval, so $F_m(h) \geq 2$. The exact 144 envelope attains two for $3 \leq h \leq 18$. Consequently for *every* budget $B \geq 144$, $G_B(h) = 2$ throughout that range, even though the all-length optimality theorem fixes $B = 256$.

Density ordering is insufficient as a general short-window rule. The worst densities satisfy $\rho_9 = 1/3 < 1/2 = \rho_4$, but $F_9(2) = 2 > 1 = F_4(2)$. Modulo 9, $n = 7$ admits adjacent positions $4, 5$. Also, keeping only maximum-density masks is invalid: the maximum-density $n = 2, 5, 8$ masks are $\{0, 3, 6\}$ and have two-window maximum one, whereas the lower-density $n = 7$ mask has maximum two. Nevertheless, in the stated global budget problem the density minimizer 144 is also the uniform window optimum. The experiment did not refute that global selection rule in this domain.

5 The finite certificate and its verification

The first construction enumerates every $m = 1, \ldots, 256$, every unit n, and every start. After forming the exact mask, it updates all window sums by the recurrence

$$W_s(0) = 0, \qquad W_s(h+1) = W_s(h) + f_{m,n}(s+h).$$

Taking the maximum over all enumerated n, s gives both an upper bound and an attaining witness for each $F_m(h)$. There is no sampling, numerical approximation, or squareclass pruning. A witness alone certifies only a lower bound; the full enumeration supplies the upper bound.

The independent implementation constructs masks from the equation $n = a^2 - b^2 \pmod{m}$, then uses repeated integer bitsets and population counts for each window. It agrees on all 16,384 original table entries, all original attaining witnesses, all budget/length tie sets, and all maximum-density residue ties.

Construction coverage	$h \leq 64$	$h \leq 146$
Unit masks	19,948	19,948
Mask predicates	3,406,801	3,406,801
Window updates	218,035,264	497,392,946
Attaining witnesses	16,384	37,376
Direct witness bit checks	532,480	2,747,136

The second implementation does not independently recompute every competing upper bound beyond 64. Instead it directly checks the analytic 144 envelope through 146 and certifies all 20,910 competing cells for $65 \leq h \leq 146$ by $\lceil h\rho_m \rceil \geq F_{144}(h)$. This suffices for optimality. It also directly checks all 37,376 extended attaining witnesses. An additional constants checker enumerates 995,328 remainder cases for $m = 144$ and confirms the excess 7/4 and the density gap.

All arithmetic used for correctness is integral or rational. Each exhaustive Python verifier and certificate checker accepts no arguments and enforces limits of 120 CPU seconds and 256 MiB of address space. Those limits apply to the Python programs, not to the Lean build or the logical mask payload. Mathematical JSON is deterministic; runtime telemetry is stored separately. Parent reruns and document-build checks are recorded in `parent_verification.json` and `SHA256SUMS`.

To audit the central result on Linux with Python 3.10 or later, run the following from the extracted package directory. No third-party Python packages are required.

```
sha256sum -c SHA256SUMS
python3 wave2_b/test_minimax_146.py
python3 wave2_b/generate_minimax_146.py
python3 wave2_b/check_all_h_constants.py
python3 wave2_c/verify_minimax.py
python3 wave2_c/verify_peer.py
python3 wave2_c/extend_optimality.py
python3 wave2_c/verify_extended_peer.py
```

The original 64-length construction can also be regenerated with `wave2_b/generate_minimax.py`. Reruns replace their result and telemetry files; preserve the supplied record by using a separate copy. The mathematical hashes are expected to agree. Hash agreement identifies artifacts and does not substitute for proof review.

6 Lean formalization and its trust boundary

The accompanying `lean/` project pins Lean 4.19.0 and the mathlib revision recorded in its package manifest. Its endpoint in `MainTheorem.lean` is

```
theorem uniform_optimal (m h : ℕ)
  (hm1 : 1 ≤ m) (hm256 : m ≤ 256) :
  F 144 h ≤ F m h
```

There is no finite-horizon hypothesis or assumed certificate-correctness premise. The statement includes $h = 0$. The same development proves strict inequality for every competing modulus when $h \geq 148$, as well as

$$12F_{144}(h) \leq h + 21, \qquad 2h \leq 21F_m(h) \quad (1 \leq m \leq 256,\ m \neq 144).$$

Connection to the mathematical definition. `Model.lean` defines the indicator by $a^2 \bmod m = (n + b^2) \bmod m$ for some $0 \leq b < m$. This equation includes zero squares and avoids truncated subtraction on natural numbers. Unit inputs satisfy $n < m$ and $\gcd(n, m) = 1$, so the modulus-one convention is included. `SemanticBridge.lean` proves that the executable window count equals the sum of this predicate, and that the executable F equals the corresponding maximum over inputs and starts. Repeated positions in windows longer than a period are counted. The formal theorem concerns these counts; the raw m-bit storage charge is the representation convention stated in Section 1, not a claim about the Lean verifier's physical memory use.

Finite checks and the infinite argument. For every unit input and start modulo 144, Lean checks the full-period upper bound, all 144 remainder bounds, and an explicit upper target at each length through 146. For each competing modulus, it checks a valid unit input, its full-period lower bound at every start, and a valid start for every remainder. It also checks a lower witness meeting the upper target in all $255 \cdot 146 = 37{,}230$ competing modulus/length pairs. Witnesses imported from Python are untrusted proposals: Lean reconstructs the masks and validates their unit conditions, starts, and count bounds. An inductively proved running-sum identity accelerates the upper check without changing its exhaustive specification.

The generic window lemmas prove, for every periodic indicator f, the identity

$$W_f(s, h) = \left\lfloor \frac{h}{m} \right\rfloor W_f(s, m) + W_f(s, h \bmod m).$$

Thus the checked period and remainder bounds imply the displayed global inequalities. Arithmetic gives the comparison from $h = 147$ onward, and the finite witnesses cover the remaining positive lengths. The proof does not need to assume the exact envelope or a Python result table.

Trust and formal scope. The finite checks use Lean's standard `native_decide` facility. The final theorem has precisely these axiom dependencies:

```
propext, Classical.choice, Quot.sound, Lean.ofReduceBool.
```

The last dependency records reliance on Lean's compiled Boolean evaluation, including its compiler and runtime; the finite computation is not performed entirely by kernel reduction. There are no `sorry` placeholders or custom mathematical axioms. The semantic and certificate-extraction proofs are checked by the kernel.

The formalization covers the central optimality theorem, eventual strict optimality, both displayed bounds, and the semantic connection to the residue model. Lemma 2, its sharpness examples, the budget table, and the historical discussion are not formalized here. Their analytic arguments and original computational evidence remain available. The formal proof does not establish historical novelty or human peer review.

Reproduction. With the pinned Lean toolchain installed through elan, run from `lean/`:

```
lake exe cache get
bash verify.sh
```

The script rechecks all nine local proof modules, reruns the finite evaluations, and prints the endpoint statements and axiom lists. The published verification record distinguishes cached dependencies from the fresh local source rebuild and records its time and memory usage. The original Python checks remain a separate verification route.

7 Prior art, novelty, and release decision

Square-difference filters and modular-hyperbola projections have substantial prior art. Bower, Evans, Luo, and Miller give CRT and prime-power projection counts [1]. Hittmeir develops the hyperbolic sieve, prime-power counts, modulus-choice examples, and a stronger growing-modulus factorization analysis [2]. Those results already cover the underlying full-period mathematics.

Hewitt's 2022 empirical study explicitly searches 792 Fermat sieve moduli formed from bounded powers of $2, 3, 5, 7$. It constructs a Pareto front for full-period reduction versus compact passing-entry table size, conditioned on 24 input categories [3]. Its tables include 144. Thus neither finite modulus optimization nor the choice of 144 is new here. The exact worst-over-unit-and-start, all-length statement under the raw 256-bit model was not located in the bounded search. That absence does not establish historical priority.

This research note and its accompanying evidence package are intended for public reproducibility and further mathematical review. The result is a verified finite-budget combinatorial theorem whose historical novelty remains unestablished. It establishes candidate-count optimality under the stated representation model, without establishing a faster factorization algorithm. Its release classification is `PUBLIC_RESEARCH`.

A further mathematical study should explicitly change one assumption at a time, for example the representation cost or modulus budget, and repeat the prior-art and certificate checks. This theorem cannot be extrapolated to those alternatives without additional proof.

References

[1] A. Bower, R. Evans, V. Luo, and S. J. Miller. *Coordinate sum and difference sets of d-dimensional modular hyperbolas*. arXiv:1212.2930v1, submitted 12 December 2012. Proposition 2.1, Lemma 2.3, Theorems 3.3 and 3.6. https://arxiv.org/abs/1212.2930v1.

[2] M. Hittmeir. *Integer Factorization as Subset-Sum Problem*. Journal of Number Theory 249 (2023), 93–118. Reviewed version: arXiv:2205.10074v1, 20 May 2022; Sections 3, 4 and 7. https://arxiv.org/abs/2205.10074v1.

[3] S. Hewitt. *Empirical explorations of faster Fermat factorisation, part 5*. Cambridge Clarion, 12 August 2022, updated 16 August 2022. Self-authored empirical study, section "A systematic approach to tuning the filter" and appendix. https://www.cambridgeclarion.org/61.html.